CipherChannel: An Application-Layer Authenticated Encryption Protocol for Secure Bluetooth Low Energy Command and Control of Medical Exoskeleton Devices

Theodoros Frixos Paparrigopoulos* , Björn Boss Henrichsen† , Marios Koniaris* , Pavlos Michalopoulos†, and Panayiotis Tsanakas*
* National Technical University of Athens, Greece
† Independent Researcher, Greece

***Abstract*—Bluetooth Low Energy (BLE) is widely used in wearable and rehabilitation devices, but BLE pairing and link-layer encryption are not sufficient as the sole protection mechanism for safety-relevant exoskeleton command channels. In a powered lower-limb exoskeleton, a replayed or injected command can trigger a physical state transition, making command authentication and replay resistance necessary above the BLE link layer. This paper presents *CipherChannel*, an application-layer authenticated encryption protocol for BLE-mediated exoskeleton command and control. Implemented above GATT, CipherChannel combines AES-256-GCM, persistent 96-bit counter nonces, direction-separated replay protection, physically gated key provisioning, and crash-safe persistent admission state, so that gateway-side cryptographic command admission gates every command before it is released to the proprietary exoskeleton runtime. We complement the protocol's security argument with a ProVerif model of the key exchange and counter-nonce procedure, machine-checking operational-key secrecy against a network attacker without access to K_T, and endpoint-bound command authentication under the stated physically gated provisioning assumptions. CipherChannel passed all protocol correctness and provisioning-gate tests, rejected all adversarial and cross-endpoint injection attempts, and achieved 169.6 ms median and 214.7 ms p99 client-observed command-to-ACK latency at approximately 1 m. A physical ESP32 cane endpoint completed 502/502 sequential encrypted command trials with 145.0 ms median and 295.2 ms p99 round-trip latency. The measurements show that crash-safe counter persistence and BLE scheduling, rather than AES-GCM computation, dominate the tested command path.**

***Index Terms*—Bluetooth Low Energy, authenticated encryption, AES-GCM, replay protection, medical exoskeletons, IoT security.**

I. Introduction

A replayed `STAND_UP` command is not just a bad packet. In a powered lower-limb exoskeleton, it can become a fall risk. This is the security distinction that motivates our work: wireless command traffic for rehabilitation robotics is not ordinary IoT telemetry. A corrupted temperature sample may degrade a dashboard; an accepted motion command can change the physical state of a machine attached to a human body.

Powered lower-limb exoskeletons have progressed from laboratory prototypes to clinically relevant rehabilitation and mobility platforms, supporting assisted standing, stepping, and ambulation for users with spinal cord injury, stroke, and other neuromotor impairments [14], [16]. Recent research systems have also explored wireless exoskeleton-control interfaces, while clinical lower-limb studies further demonstrate their relevance in rehabilitation settings [19], [20]. Bluetooth Low Energy (BLE) is an attractive technology for such interfaces because it is already available on smartphones, microcontrollers, and embedded gateways, and because GATT provides a compact abstraction for command and status exchange.

The problem is that BLE connectivity is not command authorization. BLE pairing and link-layer encryption provide useful protection, but they were not designed to be the final safety boundary for medical command traffic. Published attacks including KNOB, BIAS, BLESA, SweynTooth, InjectaBLE, and BLUFFS show that BLE deployments can be exposed to key entropy downgrade, impersonation, unauthenticated reconnection, implementation-level stack failures, encrypted-connection frame injection, and weaknesses in Bluetooth session-key derivation [1]–[6]. In other words, the BLE link may open the door, but it should not be the bouncer deciding whether a movement command reaches the exoskeleton runtime.

Application-layer security over BLE has been studied in the broader wearable and IoT literature. Prior work has proposed security frameworks between the application and GATT layers for generic BLE wearables [37] and privacy-aware authentication mechanisms for BLE-based IoT objects [38]. These studies establish the value of placing cryptographic protection above BLE pairing. However, they do not directly address the command-admission semantics of powered exoskeleton control: sparse but safety-relevant commands, replay resistance across disconnections, crash-safe replay state, physically gated provisioning, endpoint separation, and controlled forwarding from a BLE gateway into a proprietary runtime.

Conversely, wireless exoskeleton-control research has mostly emphasized control, sensing, rehabilitation performance, and communication latency rather than authenticated command admission. Existing systems demonstrate the utility of BLE or other wireless links for exoskeleton supervision, but they do not provide a reusable application-layer authenticated

encryption protocol for BLE-mediated lower-limb exoskeleton command and control [20]–[22], [42]. The missing piece is therefore not merely encryption over BLE. It is the acceptance rule: when should a gateway release a received wireless command to software that can move a device attached to a patient?

This paper answers that question with *CipherChannel*, an application-layer authenticated command-admission protocol for BLE/GATT exoskeleton control. CipherChannel accepts a command only after AEAD verification, monotonic freshness validation, direction-parity validation, and durable replay-state update. Its core design choice is to make replay-control state persistent across BLE disconnections and process restarts. This turns command admission into a crash-safe gateway decision rather than a property of the current BLE session. Although our evaluated system targets a lower-limb medical exoskeleton, the same design applies to sparse, safety-relevant BLE command channels in which replayed or modified commands may have physical consequences.

The evaluated system consists of a supervisory client, a Raspberry Pi BLE/GATT gateway, an ESP32-based auxiliary cane controller, and a proprietary lower-limb exoskeleton runtime. The supervisory client issues high-level commands such as standing, walking, and stopping. The cane acts as a second authorized control endpoint that can generate user-triggered events during operation. Both endpoints communicate with the gateway over BLE/GATT, and both must satisfy the same CipherChannel authentication, freshness, and direction checks before any command is forwarded to the runtime.

This paper makes the following contributions:

- **Threat-driven command-admission model**: We identify the gap between BLE link-layer protection and safety-relevant command acceptance in lower-limb exoskeleton supervision. The model treats replay, reflection, stale-session reuse, and command injection as first-class threats to gateway command admission.
- **CipherChannel protocol**: We present an application-layer BLE/GATT command-admission protocol using AES-256-GCM, persistent 96-bit counter nonces, direction-separated parity classes, crash-safe counter persistence, and fail-closed receive semantics. The design avoids wall-clock freshness windows, unauthenticated reset messages, and BLE-session-dependent replay state.
- **Gateway-mediated safety boundary**: We introduce an integration pattern in which BLE packets are authenticated, checked for freshness, and durably recorded before any command is released to a proprietary exoskeleton runtime. This adds a cryptographic command boundary without modifying the runtime itself.
- **Symbolic verification**: We model CipherChannel's key exchange and counter-nonce procedure in ProVerif and machine-check operational-key secrecy against a network attacker without access to (K_T), together with endpoint-bound command authentication under the stated physically gated provisioning assumptions. Replay resistance, reflection rejection, and cross-endpoint isolation are supported by the modeled accepted-counter state and the implementation's persistent-counter invariant.
- **Empirical validation**: We evaluate the reference implementation through protocol-correctness tests, provisioning-gate tests, adversarial-rejection tests, cross-endpoint injection tests, steady-state BLE command trials, cold-start trials, key-exchange trials, obstruction and range experiments, and a physical ESP32 cane hardware-in-the-loop latency experiment. The results support the replay-resistance and latency claims for sparse supervisory and auxiliary-control commands under the measured BLE conditions.

The remainder of the paper is organized as follows. Section II reviews BLE and application-layer security background. Section III presents the gateway mediated architecture, and Section IV specifies CipherChannel. Section V describes the reference implementation. Section VI reports the experimental evaluation. Section VII discusses implications, limitations, and deployment trade-offs. Section VIII concludes the paper.

II. Background and Related Work

A. BLE Security Model and Known Vulnerabilities

The BLE security architecture, defined by the Bluetooth Core Specification, combines pairing/bonding, long-term key establishment, and link-layer packet encryption. LE Secure Connections (LESC), introduced in Bluetooth 4.2, uses Elliptic-Curve Diffie–Hellman during pairing, while encrypted BLE connections protect link-layer packets using AES-CCM. Above the link layer, BLE applications interact mainly through ATT/GATT services and characteristics; although L2CAP may segment and reassemble data, GATT-level semantics and authorization remain application responsibilities [9].

Published attacks show that BLE link-layer security is not sufficient as the only protection for safety-critical command traffic. KNOB enables session-key entropy downgrade [1]; BIAS permits impersonation of previously bonded devices [2]; BLESA exploits unauthenticated reconnection [3]; InjectaBLE demonstrates frame injection into encrypted BLE connections [5]; and BLUFFS breaks forward and future secrecy by manipulating Bluetooth session-key derivation [6]. At the implementation level, SweynTooth exposed memory-safety and state-machine failures in BLE SoC stacks [4], while recent MITM experiments against modern fitness trackers show that standards-compliant devices may still remain vulnerable due to association models, standard design choices, or incomplete mitigations [40].

Surveys of BLE and BLE Mesh security reach the same conclusion: later Bluetooth versions improve confidentiality, authenticity, and privacy, but deployments remain exposed to insecure pairing, weak application-layer authorization, mobile-application attacks, fingerprinting, stack vulnerabilities, and zero-day attacks [9], [36], [39]. CipherChannel therefore treats BLE link-layer encryption as useful but insufficient and places an authenticated encryption boundary above GATT before any exoskeleton command can be accepted.

Recent IoT work has shown that AES-256-GCM can provide application-layer end-to-end protection on ESP32/Raspberry Pi pipelines, but it targets MQTT telemetry rather than replay resistant BLE command admission [12].

B. Exoskeleton Control Architectures

Lower limb exoskeleton control systems are commonly organized into a high-level intent-recognition layer, a mid level finite state machine or trajectory controller, and a low-level joint controller [14], [15]. Commercial and research lower-limb exoskeleton systems are commonly evaluated through assisted mobility tasks or training modes such as standing, sitting, stepping, walking, or gait rehabilitation [17]–[19]. These commands are sparse but safety relevant: replaying or modifying a single valid command may be more consequential than corrupting a telemetry sample.

Recent lower-limb rehabilitation robot work also adopts hierarchical distributed control architectures to improve modularity, scalability, real-time response, and control stability, reinforcing the need for well-defined command boundaries in such systems [23].

Wireless interfaces are increasingly used to move supervisory commands or sensor signals away from tethered operator stations. BLE has been used to relay smartphone inertial data for rehabilitation exoskeleton control and to evaluate communication induced latency [20]. Secure transmission has also been considered in brain controlled lower-limb exoskeleton systems [42], encrypted channels have been identified as necessary in a remotely controlled ankle-exoskeleton prototype [21], and BLE remote control has been added to a finger-exoskeleton rehabilitation platform [22]. These works establish the need for wireless exoskeleton supervision, but they do not provide a reusable application-layer BLE/GATT authenticated command channel protocol for lower-limb exoskeleton control.

C. Application Layer Security for BLE and Medical IoT

Prior work on application-layer security for medical wireless devices has focused mainly on implantable medical devices, IoMT, and healthcare-IoT systems. Command injection against a commercial neurostimulator has been demonstrated together with a lightweight session-key protocol [25]. A dual factor, smartphone proxied IMD access control scheme with forensic support has also been proposed [24]. Broader healthcare-IoT studies report continued exposure to unauthorized access, impersonation, eavesdropping, tampering, denial of service, and data integrity attacks across device, network, cloud, and application layers [41], [43], [46]. Related medical device studies further show that command manipulation, replay, and unsafe security trade-offs also occur in implantable and automated therapeutic systems [26].

Recent IoT healthcare work likewise emphasizes lightweight, device level, and edge-aware protection rather than relying only on the underlying network. Secure communication, authentication, and cryptographic mechanisms are identified as central requirements for healthcare-IoT adoption [43]. Recent studies propose lightweight access control mechanisms for smart medical wearables [44], secure data exchange requirements have been discussed for generative-AI-driven human digital twins in IoT healthcare [45], and biomedical MEMS work similarly highlights unauthorized access, tampering, interception, encryption, authentication, and monitoring as core smart-healthcare concerns [47].

Within BLE, prior work proposes an application-to-GATT security framework for wearable devices that provides fresh key generation and encryption/decryption services to mitigate replay, MITM, tampering, and passive eavesdropping attacks [37]. The tradeoff between BLE random-address privacy and authentication has also been addressed through an ECC-based access control and secure communication mechanism for BLE IoT objects [38]. Additional empirical studies of consumer BLE and Bluetooth deployments confirm persistent authentication, pairing, and protocol level weaknesses beyond the formal Bluetooth specification [10], [11], [35].

Recent BLE and embedded-IoT security work also explores alternative mechanisms for pairing resilience, key establishment, and lightweight authenticated encryption. Blockchain-backed resilient pairing and bonding has been proposed for IPv6-over-BLE device migration, combining lightweight Encrypt-then-Authenticate protection, ledger based transfer of bonding information across gateways, BAN-logic security analysis, and Deep-Q-Learning-based MITM detection [48]. Physical layer key generation combined with physically unclonable functions has also been proposed for resource-constrained IoT edge nodes, deriving session keys from BLE channel entropy and hardware-level device uniqueness without relying on a pre-shared secret [49]. The generated BLE derived keys in that approach pass the NIST Statistical Test Suite, supporting their suitability as cryptographic keying material for lightweight embedded deployments [49]. At the cryptographic implementation level, recent embedded security studies evaluate ChaCha20-Poly1305 on ARM Cortex-M4 processors as a compact, constant-time AEAD candidate for high-speed embedded IoT applications without AES hardware acceleration [50]. The reported implementation achieves substantially lower cycle counts than AES-GCM on the evaluated Cortex-M4 platform, supporting the broader point that AEAD selection depends strongly on the available embedded hardware acceleration [50].

CipherChannel differs from this prior BLE, IoMT, and healthcare-IoT work by targeting the command semantics of powered lower-limb exoskeletons. The system is built around application-layer authenticated encryption, counter based replay protection, physically gated provisioning, and controlled forwarding into the proprietary runtime. It also keeps the supervisory and cane endpoints separate. Prior above-link BLE security work establishes the value of adding cryptographic protection above pairing and link-layer encryption. Application-to-GATT security frameworks for wearable devices and ECC-based BLE access-control mechanisms provide important protection against eavesdropping, tampering, replay, and unauthorized access in generic BLE/IoT settings [37], [38]. Similarly, a DTLS- or TLS-style record layer could provide authenticated encryption above a BLE transport if an additional GATT framing and reassembly layer were implemented. CipherChannel differs in the command-admission property it targets. Rather than acting as a general secure tunnel, it binds each accepted command to a persistent endpoint-specific replay state, stores counter updates durably across

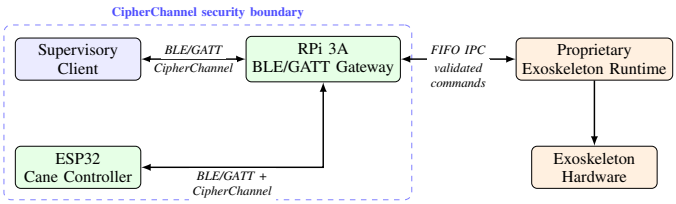

Fig. 1. Gateway mediated command boundary. CipherChannel protects BLE/GATT traffic between the supervisory client or cane and the gateway. The proprietary exoskeleton runtime receives only commands that have passed application-layer authentication, replay validation, direction validation, and persistent counter state update.

BLE disconnections and process restarts, gates operational-key provisioning through a physical action at the gateway, and forwards only validated commands to an otherwise unmodified proprietary exoskeleton runtime. Thus, the contribution is not a new AEAD primitive, but a BLE/GATT command-admission boundary for sparse safety-relevant control messages where replay acceptance after restart is itself a safety concern.

III. System Architecture

A. Gateway Mediated Command Boundary

The architecture is designed to evaluate CipherChannel as an application-layer command authentication boundary above BLE/GATT, not as a new exoskeleton control stack. The central architectural pattern is a *gateway mediated command boundary*: BLE packets terminate at a gateway, but commands are not released to the exoskeleton runtime until they have passed application-layer authentication, replay validation, direction validation, and persistent counter state update.

This separation is important because BLE connection establishment is not treated as command authorization. A peer may connect over BLE, but the gateway forwards a command only if the received CipherChannel packet is valid under the operational key, carries a fresh counter nonce, has the expected direction parity, and updates the receiver replay state before plaintext command release. The proprietary exoskeleton runtime is therefore placed behind a cryptographic admission-control point.

Fig. 1 shows the evaluated instance of this pattern. It contains four components: a supervisory client, a Raspberry Pi BLE/GATT gateway, an ESP32 auxiliary cane controller, and a proprietary exoskeleton runtime. The gateway is the only BLE GATT server visible to the supervisory client and the cane. All CipherChannel state is maintained at the gateway and at the authorized endpoints; the exoskeleton runtime remains unmodified and is unaware of the security protocol.

B. Evaluated Exoskeleton Integration

The supervisory client represents the operator facing control endpoint. In a deployment, this may be a mobile application used by a clinician or authorized user to initiate high-level operational commands such as `STAND_UP`, `START_WALKING`, `TAKE_STEP`, `STOP_ACTION`. In the experiments, the mobile application was replaced by a laptop running a Python/Bleak client to provide repeatable timing instrumentation and structured logs. This substitution changes only the BLE client implementation; the GATT writes, CipherChannel packet format, gateway validation path, and runtime forwarding logic remain the same.

The Raspberry Pi 3 Model A+ gateway terminates BLE/GATT communication and enforces the CipherChannel admission rule. It runs Raspberry Pi OS, Python 3, BlueZ, and the `bless` GATT server library. The gateway exposes command, notification, and security provisioning characteristics for the supervisory client and cane paths.

The ESP32 auxiliary cane controller represents a second authorized endpoint. It scans for the gateway service UUID, connects as a BLE central, and writes encrypted cane-trigger events to the gateway. The cane implementation uses Arduino C++ with mbedTLS and persists its CipherChannel state in ESP32 Non-Volatile Storage. A hardware button clears the stored key and triggers re-provisioning.

The proprietary exoskeleton runtime is treated as an external command consumer. It does not parse CipherChannel packets, maintain cryptographic state, or participate in BLE security. It receives only raw validated action strings from the gateway over named FIFO pipes.

C. Command Flow and Runtime Isolation

A command follows a five stage path. First, the supervisory client or cane serializes a high-level action into a compact UTF-8 payload. During experiments, the payload also includes an optional trial identifier used only for measurement:

```
{"trial_id":"t0001", "action":"STAND_UP"}
```

Second, the endpoint encrypts the payload using CipherChannel and writes the resulting `nonce || ciphertext || tag` packet to the appropriate GATT characteristic. Freshness is not encoded in the JSON payload; it is encoded in the persistent counter nonce carried in the CipherChannel packet header.

Third, the gateway receives the complete encrypted byte buffer from the BLE host stack and applies CipherChannel validation. The gateway verifies the AES-GCM tag, checks that the counter is strictly greater than the last accepted counter for that logical endpoint, checks the expected direction parity, and persists the new receive counter before exposing the plaintext command to the rest of the gateway logic.

Fourth, only after successful validation does the gateway enqueue the raw action string for the runtime-facing thread. Invalid packets are rejected before FIFO forwarding and do not produce runtime-visible commands.

Fifth, the gateway writes the validated action to the outgoing FIFO connected to the proprietary runtime. Runtime responses are read from the return FIFO and published to the appropriate BLE notification path. This preserves a narrow runtime interface: the runtime receives the same type of high-level action string it would receive without CipherChannel, while the security sensitive packet parsing and replay state remain isolated inside the gateway.

D. Endpoint Roles and State Separation

The gateway maintains separate CipherChannel contexts for the supervisory client path and the cane path. Each context has its own send counter, receive counter, direction parity, and replay state. This prevents a valid packet from one logical endpoint path from being accepted as fresh traffic on another path. The separation is particularly important because the supervisory client and the cane can both communicate with the same gateway during the same operational window.

Operationally, the supervisory client initiates high-level mode changes, while the cane may later continue or interrupt execution through user triggered events. Both paths are subject to the same command admission rule: a command is forwarded to the runtime only after CipherChannel authentication, freshness validation, direction validation, and replay state persistence.

IV. The CipherChannel Protocol

A. Threat Model

We adopt the following adversarial model for the wireless command channel between the supervisory client / cane controller and the RPi gateway.

a) In-scope threats:

- **Passive eavesdropping**: An adversary captures BLE radio frames.
- **Replay attack**: A captured valid ciphertext is retransmitted to trigger an unintended exoskeleton movement.
- **Message modification**: An intercepted ciphertext is altered before delivery, attempting to corrupt or substitute a command.
- **Session hijacking / MITM**: An adversary attempts to inject frames into an established BLE session (cf. InjectaBLE [5]).
- **Unauthorized command injection without endpoint secrets**: An adversary that does not possess the endpoint's operational key $K_{S,e}$ or the bootstrap transport key K_T attempts to inject, modify, replay, reflect, or reorder command packets over the BLE/GATT channel.
- **Stale-session attack**: A client that has lost synchronization with the gateway attempts to re-use an expired cipher context.

b) Out-of-scope threats: Physical compromise of the gateway hardware, side channel attacks on the cryptographic implementation, and denial-of-service attacks against the BLE radio are considered out of scope. Compromise of an authorized endpoint before command generation (e.g., capturing valid encrypted packets from a compromised client for later replay) is also out of scope; CipherChannel assumes the endpoint is trusted at the time of command creation. Authentication of the user or clinician operating the endpoint is handled by higher level policies and is not part of the protocol. Denial-of-service attacks against BLE availability, including radio jamming, connection-slot exhaustion, repeated unauthorized connection attempts, and high-rate invalid GATT writes, are outside the cryptographic security guarantee of CipherChannel. The protocol guarantees that such traffic is not accepted as a valid exoskeleton command unless it authenticates under the operational key and satisfies freshness and direction checks. Availability and fail-safe behavior—for example, a watchdog that commands a safe stop on link loss so that a suppressed `STOP_ACTION` does not leave the device in an unsafe state—are treated as system-level safety controls layered around the cryptographic boundary, rather than as guarantees of the command-admission protocol itself. Compromise or disclosure of K_T, compromise of an authorized endpoint, and malicious behavior by an endpoint that legitimately holds K_T are outside the cryptographic threat model. In particular, because the evaluated provisioning design uses a shared bootstrap key, K_T is treated as a deployment secret installed only on trusted endpoints. Endpoint identity in provisioning associated data binds acceptance of provisioning messages to the intended endpoint, but does not provide confidentiality of one endpoint's transported operational key against another authorized endpoint that already holds K_T.

B. Cryptographic Primitives

CipherChannel is built on AES-256-GCM, a nonce-based authenticated encryption with associated data (AEAD) scheme specified by NIST SP 800-38D [31]. AEAD is required for the command channel considered here because confidentiality alone is insufficient: the receiver must also authenticate the origin and integrity of each command before releasing it to the exoskeleton runtime. The 128-bit GCM authentication tag provides integrity protection for the short command payloads used by the gateway.

AES-GCM was selected because it is standardized, widely implemented in embedded and server-side cryptographic libraries, and provides confidentiality, integrity, and authenticity in a single primitive. Prior embedded security studies have also shown that AES-family primitives remain practical on low-end IoT hardware and Raspberry Pi-class platforms, although performance depends strongly on hardware acceleration, implementation language, and cryptographic-library support [33], [34]. This choice is not intended to imply that AES-GCM is always the fastest AEAD scheme on every embedded platform. On software only embedded targets without hardware AES acceleration, ChaCha20-Poly1305 [50] has been reported to outperform AES-GCM substantially; for example, De Santis and Sigl report a 12.9× cycle-count advantage over AES-GCM on an ARM Cortex-M4 platform [50]. This comparison is relevant for microcontrollers without AES acceleration, but it does not directly determine the bottleneck in the evaluated CipherChannel path.

In CipherChannel, command payloads are short, typically consisting of compact JSON action objects. Under the tested conditions, AES-256-GCM authentication and decryption were not the dominant source of command latency; BLE transport scheduling and durable receive counter persistence contributed more to the measured command path, as shown in Section VI. Therefore, while ChaCha20-Poly1305 remains a reasonable alternative for platforms without AES acceleration, AES-256-GCM is suitable for the evaluated Raspberry Pi gateway path and for embedded targets that provide efficient AES support [50].

For CipherChannel, the primary cryptographic requirement is nonce uniqueness under a fixed operational key. The persistent 96-bit counter nonce construction in Section IV-E is designed to satisfy this requirement across disconnections and process restarts. The AEAD primitive may be revisited for future ports whose hardware accelerators, cryptographic libraries, or certification requirements differ from the evaluated implementation.

A standard secure transport alternative would be to tunnel a DTLS-style record layer over an application-defined BLE/GATT framing layer. We did not use DTLS as the primary design basis because CipherChannel targets command admission rather than general-purpose secure transport. In the evaluated setting, a BLE reconnection does not create a new CipherChannel session; the endpoint reloads its operational key and persistent counter state and can resume command validation without an additional cryptographic handshake. By contrast, in the no-resumption DTLS cold-start configuration evaluated in Section VI-F, application data can be exchanged only after explicit DTLS session establishment.

DTLS also does not remove the main state-management requirement of this work. DTLS record replay protection is tied to transport-session epoch and sequence-number state, whereas the safety requirement considered here is persistent command admission state across BLE disconnections and process restarts. A DTLS-based design would therefore still need application-managed durable state, or an equivalent recovery mechanism, to avoid accepting stale commands after endpoint or gateway restart.

Finally, BLE/GATT is exposed to applications through characteristic writes, reads, notifications, ATT MTU limits, and host-stack buffering rather than through a native datagram socket abstraction. A DTLS over GATT design would therefore require an additional framing and reassembly layer before the DTLS record layer could be used. CipherChannel instead places the authenticated command frame directly at the GATT payload boundary and performs AEAD verification, freshness validation, and receive counter persistence before command release.

C. Packet Format

Each CipherChannel packet transmitted over a BLE GATT write or notification has three fields:

$$\underbrace{\text{CtrNonce}}_{12\,\text{B}} \,\|\, \underbrace{\text{Ciphertext}}_{n\,\text{B}} \,\|\, \underbrace{\text{Tag}}_{16\,\text{B}}$$

where *CtrNonce* is a 96-bit little-endian counter value used as the AES-GCM nonce, *Ciphertext* is the encrypted application payload, and *Tag* is the 128-bit GCM authentication tag. Freshness is encoded directly in the counter nonce rather than in plaintext sequence or timestamp fields. The nonce is public, unique, monotonically increasing per direction, and authenticated by AES-GCM as part of tag verification. The protocol uses the conventional 96-bit AES-GCM IV size recommended for the efficient GCM construction [31]. After even/odd direction coding, each direction still has a usable nonce space of 2^{95} packets per operational key, which is far beyond the lifetime command volume of the target exoskeleton deployment.

The plaintext protected by AES-GCM is a UTF-8 encoded application payload. In the evaluated implementation, this payload is a compact JSON command object containing the requested action and, during experiments, an optional trial identifier used only for measurement. A representative payload is {`"trial_id":"t0001", "action":"STAND_UP"`}. The protocol does not rely on JSON-specific semantics: CipherChannel treats the encoded payload as an opaque byte string and provides confidentiality, integrity, and replay protection for the complete byte sequence.

AES-GCM supports arbitrary-length plaintexts and therefore does not require block padding. Consequently, the CipherChannel packet contains only the 96-bit counter nonce, the ciphertext corresponding to the encoded payload, and the 128-bit authentication tag. Freshness is not stored inside the JSON object; it is provided by the persistent counter nonce carried in the packet header. Because AES-GCM ciphertext length equals plaintext length, a small fixed command vocabulary makes command *content* confidential while still leaking a coarse indication of command *identity* through ciphertext length; where identity confidentiality is required, fixed-length plaintext padding is a straightforward mitigation that does not alter the wire format semantics (see Section VII-E).

D. Handling BLE MTU Constraints

A CipherChannel packet has 28 bytes of fixed cryptographic overhead: 12 bytes for the counter nonce and 16 bytes for the AES-GCM authentication tag. Freshness is encoded in the persistent 96-bit counter nonce, so the packet does not carry separate plaintext sequence-number or timestamp fields. With the default BLE ATT MTU of 23 bytes, approximately 20 bytes are available for the attribute value after ATT headers. Therefore, even short encrypted commands may exceed the payload size of a single default-MTU GATT write.

The evaluated implementation passes the complete CipherChannel frame to the BLE host stack and relies on the behavior provided by the client library, BlueZ, and the GATT implementation, including long write handling or stack-level segmentation where available. This design choice does not affect the cryptographic validation rule. The gateway applies CipherChannel processing only after receiving a complete encrypted byte buffer. The buffer is then parsed as `CtrNonce || Ciphertext || Tag`; AES-GCM authentication is verified; counter monotonicity and direction parity are checked; the accepted receive counter is persisted; and only then is the plaintext command forwarded to the runtime interface.

For BLE stacks that do not transparently carry frames larger than the effective ATT payload, CipherChannel can use a simple application level fragmentation profile above GATT. In this profile, the sender first constructs the complete CipherChannel frame and then splits it into fragments. Each fragment carries a small plaintext reassembly header, for example `frame_id`, `frag_index`, `frag_count`, and `total_len`, followed by a fragment of the opaque CipherChannel frame. The receiver buffers fragments with the

same `frame_id`, rejects duplicate or out-of-range fragment indexes, enforces the configured maximum frame size, and discards incomplete reassembly state after a timeout. No fragment is decrypted or released to the application independently; AES-GCM verification, replay validation, direction validation, and receive counter persistence are performed only after the full CipherChannel frame has been reassembled.

To characterize the evaluated BLE path, we performed an MTU/long write confirmation experiment from the laptop/Bleak client to the Raspberry Pi/BlueZ gateway. The backend exposed a negotiated ATT MTU of 23 bytes, but the evaluated BlueZ/Bleak path still accepted complete CipherChannel frames larger than the default 20-byte ATT value. Encrypted frames succeeded up to a 400-byte plaintext command, corresponding to a 428-byte CipherChannel frame, while a 512-byte plaintext command, corresponding to a 540-byte encrypted frame, failed.

This result confirms that the evaluated stack can carry the command sized encrypted frames used in our experiments. It should not be interpreted as a portable guarantee across BLE stacks. The fragmentation profile above provides a compatibility path for heterogeneous mobile operating systems, microcontroller BLE stacks, or larger payloads, but it was not exercised in the reported evaluation.

E. Persistent Counter Nonces and Replay Protection

CipherChannel uses the AES-GCM nonce field to carry the freshness counter used for replay control. Each endpoint maintains two persistent 96-bit counters: one for messages it sends and one recording the highest valid counter it has received from its peer. The state file contains the operational key and both counters and is updated atomically using a temporary file followed by `os.replace()`, so a crash cannot leave a partially written state file.

Algorithm 1 summarizes the send and receive procedures used by CipherChannel to construct counter nonces, reject stale or reflected packets, and admit only successfully authenticated commands.

The two traffic directions are separated by counter parity. During channel creation, exactly one endpoint is marked as the initiator. The initiator sends only even counter values, while the non-initiator sends only odd counter values:

$$ctr_{send} \leftarrow ctr_{send} + 2, \qquad nonce = ctr_{send}. \tag{1}$$

The send counter is persisted before the encrypted packet is returned to the caller. This ordering prevents nonce reuse after a crash between packet construction and physical transmission. Reusing a GCM nonce under the same key would violate AES-GCM security, so this write-before-send rule is a security critical implementation requirement.

If authentication succeeds, it interprets the nonce as a little-endian integer c and accepts the message only if the counter is fresh and belongs to the expected receive-direction parity class:

$$c > ctr_{recv} \quad \wedge \quad (c \bmod 2) = parity_{recv}. \tag{2}$$

Algorithm 1 CipherChannel Send/Receive Procedure

Input: Operational key K_S; persistent state $(ctr_s, ctr_r, parity_s, parity_r)$
Output: Valid packets are released once; invalid or replayed packets return $\perp$

function SEND($plaintext$)
  $ctr_s \leftarrow ctr_s + 2$
  if $ctr_s \geq 2^{96}$ **then**
    return provisioning required
  end if
  Persist ctr_s for the current key version and channel
  $nonce \leftarrow$ encode ctr_s as a 12-byte little-endian value
  $(ciphertext, tag) \leftarrow \text{AESGCM.ENC}_{K_S}(nonce, plaintext)$
  return $nonce \| ciphertext \| tag$
end function

function RECEIVE($nonce \| ciphertext \| tag$)
  $counter \leftarrow$ decode $nonce$ as a 96-bit little-endian integer
  if $counter \leq ctr_r$ **then**
    return $\perp$ ▷ replay or stale packet
  end if
  if $(counter \bmod 2) \neq parity_r$ **then**
    return $\perp$ ▷ wrong direction or reflection
  end if
  $plaintext \leftarrow \text{AESGCM.DEC}_{K_S}(nonce, ciphertext, tag)$
  if $plaintext = \perp$ **then**
    return $\perp$ ▷ tampering or wrong key
  end if
  Persist $ctr_r \leftarrow counter$ for the current key version and channel
  return $plaintext$
end function

The receiver then persists c as the new highest received counter *before* returning the plaintext command to the gateway logic. This ordering prevents replay after a crash occurring between successful decryption and command execution. A replayed packet has $c \leq ctr_{recv}$ and is therefore rejected. A packet reflected back to its sender has the wrong parity and is also rejected. A packet modified in transit fails GCM tag verification before any counter update is attempted.

The counter nonce construction does not require wall clock stamps, timing windows, sliding replay windows, or an unauthenticated reset transition. Missed packets do not cause permanent desynchronization because the receiver accepts any higher same-parity counter. The protocol does not, however, acknowledge delivery: an attacker can still suppress BLE packets or cause denial of service. Delivery confirmation and retry policy therefore remain application-layer responsibilities.

Counter state must never be reset under the same operational key. Reinitializing counters to zero is safe only when a fresh operational key is generated during a new provisioning event. If the same operational key is reused, the endpoint must load the existing persistent state rather than recreate the channel.

Replay resistant authentication and lightweight rekeying have also been studied for healthcare BLE and IoT-edge settings, further supporting the need for freshness and key lifecycle mechanisms above the wireless link layer [13].

F. Security Invariant

For a fixed operational key and logical channel, CipherChannel ensures nonce uniqueness, replay rejection, reflection rejection, and modification/injection resistance, assuming correct persistent-state handling and standard AES-GCM security. Nonce uniqueness follows because the send counter is increased by two and persisted before packet transmission, while

the two traffic directions use disjoint parity classes. Replay resistance follows because a receiver accepts only counters that are strictly greater than the last persisted receive counter and stores the new counter before exposing the plaintext command to the gateway. Reflection attacks are rejected because packets generated in one direction have the wrong parity when injected into the opposite receive path. Finally, packet modification or injection succeeds only if the adversary can forge a valid AES-GCM tag under the operational key. Thus, an attacker who observes, replays, reorders, reflects, or modifies BLE frames cannot cause the gateway to accept an unauthorized command except with negligible AES-GCM forgery probability.

G. Dual Key Architecture

CipherChannel separates bootstrap authorization from operational command protection. The protocol uses two classes of key material:

- **Transport Key** (K_T): a static 256-bit pre-shared bootstrap key installed at provisioning time on the gateway and on authorized controller endpoints. K_T is used only to protect the operational key during the provisioning exchange in Section IV-H; it is not used for normal exoskeleton commands. Although the key value is shared, each endpoint's provisioning traffic under K_T is a separate, persistent CipherChannel context with its own send and receive counters, so nonce uniqueness under K_T is enforced per endpoint (Section IV-H) rather than relying on a single shared counter.
- **Operational Keys** ($K_{S,client}$ and $K_{S,cane}$): endpoint-specific 256-bit keys used for authenticated encryption of operational command traffic. Each authorized endpoint obtains its own operational key during a physically gated exchange. The gateway maintains separate cipher contexts for the client path and the cane-controller path; each context has its own operational key, send counter, receive counter, parity role, and replay state.

This separation ensures that BLE pairing success is not sufficient for command authorization. An accepted command must be protected under the endpoint-specific operational key for its logical channel and must satisfy AEAD verification, counter monotonicity validation, and direction parity checks. Here, *endpoint isolation* denotes fault and replay isolation—separate keys, counters, and parity classes, so that a valid packet or key in one endpoint context is never accepted as fresh in another—rather than confidentiality of one endpoint's operational key against another authorized endpoint that also holds K_T (Section IV-H). The packet format is unchanged by endpoint-specific keying because key selection is determined by the GATT characteristic and logical endpoint context rather than by an additional packet field.

H. Physically Gated Key Exchange

Key exchange is the process by which a client acquires its endpoint-specific operational key $K_{S,e}$ from the gateway over BLE. The protocol is described in Algorithm 2.

A provisioning event that initializes counters always installs a fresh operational key version. Counter state must never be reset under an existing operational key. If an endpoint already has a valid operational key and no fresh provisioning event is completed, it must reload the stored key version and persistent counters. This rule is required for AES-GCM nonce uniqueness: counter reinitialization is safe only when paired with a new independent operational key.

Algorithm 2 CipherChannel Key Exchange Protocol

Input: Client and gateway share K_T; gateway can generate a fresh endpoint-specific operational key $K_{S,e}^{(v)}$ for endpoint e

Output: Client acquires $K_{S,e}^{(v)}$, encrypted and endpoint-bound under K_T

1: **Gateway:** Operator physically presses button on gateway hardware
2: **Gateway:** $button\_pressed \leftarrow$ True; start provisioning timer τ (configured to 60 s)
3: **Client:** Write `REQUEST_KEY` to endpoint e's Security characteristic
4: **if** $button\_pressed$ and τ not expired **then**
5: **Gateway:** Generate fresh $K_{S,e}^{(v)} \leftarrow \{0,1\}^{256}$ and assign key version v
6: **Gateway:** $C_e^{(v)} \leftarrow \text{CipherChannel}_{K_T}.\text{encrypt}(K_{S,e}^{(v)}, v;\ \text{aad} = e)$
7: **Gateway:** Write $C_e^{(v)}$ to endpoint e's Security characteristic
8: **Client:** $K_{S,e}^{(v)} \leftarrow \text{CipherChannel}_{K_T}.\text{decrypt}(C_e^{(v)};\ \text{aad} = e)$; discard on authentication failure
9: **Client:** Store $K_{S,e}^{(v)}$, key version v, and initialize persistent counters for the fresh key version only
10: **Client:** Write `STOP_SHARING` to Security characteristic
11: **Gateway:** Overwrite Security characteristic with $\text{CipherChannel}_{K_T}.\text{encrypt}(\texttt{WAITING})$
12: **Gateway:** $button\_pressed \leftarrow$ False
13: **else**
14: **Gateway:** No response; key not transmitted
15: **end if**

The exchange protects the operational key K_S by encrypting it under AES-256-GCM with the temporary provisioning key K_T. The encrypted response is exposed only during a physically gated provisioning window, and the security characteristic is cleared after `STOP_SHARING`. Incomplete exchanges are closed by timeout, and fresh counter state is initialized only together with a fresh operational key; otherwise, stored counters are reloaded to avoid AES-GCM nonce reuse.

Because K_T is shared by authorized endpoints, confidentiality under K_T alone is insufficient to bind a provisioning response to the intended endpoint. CipherChannel therefore authenticates the endpoint identity e as AES-GCM associated data during provisioning. A response generated for one endpoint verifies only when the decrypting client supplies the same endpoint identity, preventing a client provisioning response from being accepted as a cane response, or vice versa. This binding is carried in the authentication tag and does not add fields to the wire format.

We make the trust boundary of this construction explicit. Because K_T is shared among authorized endpoints, the endpoint-identity AAD binds *acceptance* but does not by itself provide *confidentiality* of $K_{S,e}$ against another party that already holds K_T: AES-GCM is a counter-mode construction whose keystream depends on the key and nonce but not on the associated data, so an endpoint holding K_T that observes a provisioning ciphertext for another endpoint could in principle recover the transported operational key. This is consistent with the threat model in Section IV-A, which treats authorized endpoints as trusted at provisioning time, and with the operational requirement that provisioning

is performed in a controlled setting during a physically gated window rather than during normal operation. Confidentiality of the operational keys therefore rests on these trust and physical-presence assumptions; replacing the static K_T with per-endpoint or ephemeral authenticated key establishment (Section VII-E) would remove this dependence. The security of CipherChannel does not rely on concealing the existence of the provisioning exchange from a network observer.

Nonce management for K_T-protected traffic is handled using the same persistent counter-nonce discipline as operational traffic. The gateway maintains an independent long-lived K_T CipherChannel context for each endpoint, with separate persistent send and receive counters. These counters survive process restarts and advance across provisioning sessions, independently of the newly generated operational key issued in a particular exchange. Thus, provisioning packets are subject to the same monotonic-counter and direction-parity checks as operational packets.

The provisioning window limits exposure of the key-transfer characteristic: it is bounded, accepts only one completed exchange, and is cleared after `STOP_SHARING`. However, replay resistance does not rely on the timeout or on characteristic clearing alone. A captured provisioning ciphertext is rejected because its counter is no longer fresh for the corresponding endpoint's persistent K_T channel.

I. Reconnection and State Recovery

CipherChannel does not define an unauthenticated counter reset command. Ordinary packet loss, BLE disconnection, and reconnection do not require a protocol reset because each endpoint reloads its persistent counter state and continues from the stored values. A replayed pre-disconnection packet is rejected because its counter is not strictly greater than the stored receive counter, while a reflected packet is rejected because it has the wrong direction parity. If an endpoint loses or corrupts its persistent state, the secure recovery path is to perform a new physically gated provisioning event that installs a fresh operational key and fresh counters. Recreating counters under the same key is forbidden because it could repeat an AES-GCM nonce.

J. Symbolic Protocol Verification

As a complement to the informal security argument in Section IV-F, we modeled CipherChannel in ProVerif 2.05 [32] under the standard Dolev–Yao attacker model. The model covers the endpoint-specific key exchange in Algorithm 2 and the counter-nonce send/receive procedure in Algorithm 1 for both the supervisory-client and cane-control paths. The ProVerif source is included in the companion repository.

AES-GCM is represented by the standard symbolic abstraction of authenticated encryption: ciphertexts reveal no plaintext without the corresponding key, and decryption succeeds only for ciphertexts produced under the same key, nonce, plaintext, and associated data. Since ProVerif does not reason directly about integer inequalities, monotonic freshness is represented by per-endpoint accepted-counter state; this preserves the property checked by the model, namely that an already accepted counter cannot be accepted again. The model assumes each endpoint's counter sequence under K_T is itself fresh across sessions, which the reference implementation provides by keeping one persistent CipherChannel context per endpoint for K_T-protected traffic (Section IV-H) rather than reinitializing counter state on each provisioning exchange. Direction separation is represented by distinct symbolic direction tags, corresponding to the even/odd counter-parity rule used in the implementation. The physically gated provisioning step is represented as a private channel unavailable to the network attacker; this abstraction models the controlled, physically gated provisioning window described in Section IV-H, and, consistent with that section, it captures secrecy against a *network* attacker rather than against another authorized endpoint holding K_T. The wall-clock timeout of the provisioning window is not modeled because it is a liveness constraint rather than a secrecy or correspondence property.

The verification establishes secrecy of the endpoint-specific operational keys against a network attacker that does not possess K_T, together with correspondence between authenticated client transmission and gateway command acceptance under the stated provisioning assumptions. In particular, the gateway acceptance event is reachable only for commands protected under the correct endpoint key, nonce state, direction binding, and associated data. The model therefore excludes replay, reflection, wrong-key acceptance, and cross-endpoint command acceptance within the symbolic abstraction. It also confirms that relaying provisioning traffic between the phone and cane paths does not cause one endpoint to adopt the other endpoint's operational key when endpoint identity is bound as associated data.

ProVerif did not discharge the injective form of the correspondence automatically, although it reported no attack trace. This is consistent with ProVerif's known incompleteness for injective correspondences involving replicated processes and mutable state. We therefore use the ProVerif result as machine-checked support for key secrecy and non-injective authentication, while replay freedom is argued from the explicit accepted-counter state in the model and the persistent monotonic-counter invariant in Section IV-F.

V. Reference Implementation

The reference implementation includes a Python/PyCryptodome gateway implementation and an ESP32/mbedTLS cane-endpoint implementation. The packet format is library-independent and is defined by the `nonce || ciphertext || tag` layout, the 96-bit counter nonce, and the receive-side monotonicity and direction parity checks. The reference implementation and experiment artifacts are publicly available under the Apache-2.0 license at https://github.com/thodpap/CipherChannel.

A. Implementation Robustness

The implementation treats CipherChannel state as security critical shared state rather than as ordinary application metadata. On the Raspberry Pi gateway, cryptographic contexts,

send counters, receive counters, and endpoint-specific key material are protected by Python locks so that BLE callbacks, runtime forwarding threads, and notification paths cannot concurrently mutate replay state. On the ESP32 cane endpoint, the corresponding state transitions are guarded by FreeRTOS synchronization primitives. This prevents concurrent command handling from producing duplicate send counters, stale receive counter updates, or inconsistent endpoint state.

The gateway rejects malformed packets before runtime forwarding and does not enqueue unauthenticated traffic. In a deployment hardened against availability attacks, the same boundary should be combined with per-connection invalid-packet counters, disconnect-on-threshold behavior, bounded command queues, throttled logging, maximum frame-size checks, and cooldowns for repeated unauthorized writes.

Crash-safe persistence is enforced through ordered state updates. Send counters are made durable before encrypted packets are returned for transmission, preventing AES-GCM nonce reuse after a crash between packet construction and physical delivery. Receive counters are made durable before plaintext commands are released to the gateway logic, preventing a post crash replay of an already accepted packet. On the Python gateway, state updates are written to a temporary file, the file buffer is flushed, the temporary file is forced to stable storage using `fsync`, and the result is committed with atomic rename via `os.replace()`. This ordering is deliberately conservative: it adds measurable latency, but preserves the replay-safety invariant across process restarts.

The implementation also isolates keys and replay state per logical endpoint. The supervisory client path and the cane controller path use separate operational keys, counters, direction parity roles, and receive state records. As a result, a valid packet from one endpoint context is not accepted as fresh traffic in another endpoint context, even when both communicate with the same gateway during the same operational window.

The cross-platform implementation was designed to keep the protocol independent of a single language or cryptographic library. The gateway path uses Python and PyCryptodome on Raspberry Pi/BlueZ, while the cane endpoint uses Arduino C++ and mbedTLS on ESP32 with non-volatile storage for persistent state. The evaluation uses the Python/Bleak client and Python/Raspberry Pi gateway path for the main BLE latency, adversarial-rejection, cold-start, key-exchange, and distance measurements.

VI. Experimental Evaluation

The evaluation is organized around four research questions.

- **RQ1 – Security rejection**: Does CipherChannel reject replay, ciphertext/tag tampering, stale counters, wrong keys, reflection, reordering, and cross endpoint injection before a command reaches the runtime?
- **RQ2 – Persistence and provisioning**: Does the protocol preserve replay state through simulated restart and enforce the physically gated provisioning state machine?
- **RQ3 – Measured command latency**: Under the tested BLE conditions, what latency is observed for steady state command-to-ACK exchange, cold start operation, key exchange, distance/obstruction scenarios, and the physical ESP32 cane endpoint?
- **RQ4 – Local processing cost**: Is the command path dominated by BLE transport, cryptographic processing, or durable counter persistence?

The experiments deliberately separate protocol validation from BLE transport. Unit and adversarial tests exercise the CipherChannel state machine without relying on radio timing. BLE command path experiments measure the latency observed by a client communicating with the Raspberry Pi gateway. Gateway internal measurements instrument the receive callback to separate packet parsing, freshness validation, AES-GCM authentication/decryption, persistent counter update, JSON parsing, and ACK publication.

A. Experimental Setup

The client ran on a consumer x86 laptop (Fedora Linux, Python 3.14, PyCryptodome 3.23.0, Bleak 3.0.2, BlueZ 5.86) with a USB BLE adapter; the gateway ran on a Raspberry Pi 3 Model A+ (BlueZ 5.86, bless 0.3.0). Full environment specifications are in the companion repository. Persistent CipherChannel state on the gateway was stored in the gateway's configured local persistent state directory and updated using synchronous file durability before command release. The persistent-storage medium was not varied experimentally; consequently, the receive-counter persistence results in Table V should be interpreted as measurements of this gateway/storage configuration rather than as storage-independent constants. A Python/Bleak client was used instead of the mobile application in the main latency experiments in order to obtain repeatable timing measurements over a large number of trials, while preserving the same GATT write path, CipherChannel packet format, gateway validation logic, and command forwarding behavior.

Because the command-to-ACK median is dominated by BLE scheduling and gateway-side counter persistence (Tables II and V), both of which are not determined by client CPU speed, the Python/Bleak client is suitable for characterizing the tested gateway-dominated path. The negotiated BLE connection parameters were not directly logged in this experimental run, so the absolute BLE scheduling component should be treated as implementation- and stack-specific. On mobile central stacks these parameters are governed by operating-system-specific policy and may shift the absolute median, whereas the gateway-side processing in Table V, where CipherChannel's own cost is incurred, is unchanged by the choice of client.

The maximum plaintext command size was 400B, producing a maximum CipherChannel packet size of 428B after adding the 12-byte counter nonce and 16-byte AES-GCM authentication tag. Unless otherwise noted, reported latency values and percentiles were computed across the recorded trials using linear-interpolation percentiles (Hyndman–Fan Type 7). For the laptop/Raspberry Pi latency tables, Tables II–VIII additionally report a 95% confidence interval obtained by percentile bootstrap (10,000 resamples with replacement over the per-trial measurements). For the ESP32 hardware-in-the-loop results, we report percentiles without bootstrap

intervals because the experiment consists of a single physical endpoint run; bootstrap intervals would capture only within-run sampling variability, not endpoint-to-endpoint or run-to-run variability.

The canonical BLE command path evaluation covers the Python/PyCryptodome implementation, the Raspberry Pi BLE gateway path, the provisioning-gate state machine, and BLE client-to-gateway command exchange. The additional ESP32 hardware-in-the-loop experiment covers physical cane-to-gateway interoperability and cane-originated round-trip timing. Physical Raspberry Pi power-cycle recovery, long-duration soak behavior, mobile-application latency, and clinical workflow execution remain outside the present evaluation.

B. Security Analysis Against Known BLE Attacks

The BLE attacks discussed in Section II motivate the need for a command authentication boundary above the BLE link layer. CipherChannel does not attempt to repair the underlying BLE protocol or make the BLE stack immune to downgrade, impersonation, reconnection, injection, or implementation-level attacks. Instead, it reduces the security dependence placed on BLE by requiring every command to satisfy an independent application-layer acceptance rule before it is forwarded to the exoskeleton runtime.

Attacks such as KNOB [1] and other downgrade attacks [7], BIAS [2], BLESA [3], SweynTooth [4], InjectaBLE [5], BLUFFS [6], and pairing downgrade or method-confusion attacks [8] affect different parts of the Bluetooth security model, including key negotiation, peer authentication, reconnection, frame injection, session-key derivation, and BLE stack robustness. In the CipherChannel design, however, BLE connection state is not sufficient for command authorization. A received packet is accepted only if it authenticates under the operational key, carries a strictly fresh counter nonce, satisfies the expected direction parity, and causes the persistent receive counter to be updated before plaintext command release.

This design means that a weakened or spoofed BLE connection may still deliver traffic to the gateway, but it cannot by itself produce a valid exoskeleton command. Tampered packets fail AES-GCM authentication, replayed packets fail the monotonic counter check, reflected packets fail the direction parity rule, and packets generated without the operational key fail tag verification. The remaining residual risks are denial of service, BLE stack crashes, physical compromise, endpoint compromise before encryption, and platform-level implementation vulnerabilities. These risks are outside the cryptographic acceptance rule and require additional system hardening, monitoring, and deployment controls.

C. Protocol Correctness and Provisioning Gate

The protocol correctness suite passed all 52 tests. The suite covered four groups: packet-length invariants (15 tests), counter behavior (21 tests), malformed-input rejection (13 tests), and protocol test-vector validation (3 tests). These tests confirmed that CipherChannel adds exactly 28B of packet overhead for plaintext sizes from 0 to 400,B, that ciphertext length equals plaintext length, that the nonce is encoded as a 12-byte little-endian counter, that direction parity is enforced, and that the counter-exhaustion guard triggers before nonce reuse. The provisioning-gate suite also passed all 28 tests, covering closed-gate rejection, active-window acceptance for the intended endpoint, wrong-endpoint rejection during an active window, one-shot provisioning, timeout behavior, gate closing after successful provisioning, re-opening after a completed exchange, independent phone/cane gates, and concurrent provisioning attempts. These are state-machine tests; physical button timing and GPIO behavior were not measured in this run.

TABLE I
ADVERSARIAL-REJECTION TEST RESULTS

Case	Att.	Acc.	Rej.	Unexp.	Interpretation
Exact replay, local	100	0	100	0	Previously accepted ciphertexts were rejected.
Exact replay, BLE end-to-end	30	0	30	0	Replay was rejected over the measured BLE path.
Replay after BLE reconnect	30	0	30	0	Reconnect did not reset receive counter state.
Replay after simulated process restart	30	0	30	0	Reloaded state rejected post-restart replay.
Ciphertext/tag tampering	120	0	120	0	Modified packets failed authentication.
Wrong operational key	4	0	4	0	Wrong-key traffic did not authenticate.
Cross-endpoint injection	60	0	60	0	Packets from one endpoint context were rejected on the other endpoint path.
Direction reflection	100	0	100	0	Wrong parity rejected reflected packets.
Reordering $2\rightarrow6\rightarrow4\rightarrow8$	120	90	30	0	Fresh packets accepted; stale packet 4 rejected.
Total	**594**	**90**	**504**	**0**	No unexpected command acceptance.

D. Adversarial Rejection and Endpoint Isolation

Table I summarizes the adversarial-rejection and endpoint-isolation suite. Across 594 replay, tampering, stale-counter, wrong-key, cross-endpoint injection, reflection, and reordering attempts, CipherChannel produced zero unexpected acceptances. The reordering case confirms the intended monotonic counter semantics: packets with counters 2, 6, and 8 were accepted as fresh, whereas a later delivery of counter 4 after counter 6 was rejected as stale.

Endpoint isolation was also validated by injecting packets generated for one logical endpoint into the other endpoint path. All 60 cross-endpoint injections were rejected before runtime forwarding, while one valid command per channel was accepted as a sanity check. This confirms that endpoint-specific keys, replay state, and direction parity expectations prevent packets from one endpoint context from being accepted in another.

Together, these tests show that invalid packets are rejected deterministically at the protocol layer, while range-related failures observed in later experiments appear as BLE connection, write, notification, or ACK-read failures rather than ambiguous CipherChannel accept/reject behavior.

TABLE II
STEADY STATE SECURE COMMAND WRITE AND TIMING-ACK LATENCY FOR STAND_UP (MS, $N = 1000$)

Phase	μ	p50	95% CI (p50)	σ	p95	p99
Client encrypt	0.232	0.215	[0.207, 0.232]	0.058	0.354	0.408
BLE write	80.231	79.245	[79.218, 79.264]	6.645	80.401	124.307
ACK wait	90.227	90.094	[90.078, 90.117]	2.543	91.147	91.531
Command→ACK	170.713	169.596	[169.569, 169.619]	7.068	170.743	214.709

TABLE III
COLD START PROVISIONING AND FIRST-COMMAND LATENCY (MS, $N = 500$)

Phase	μ	p50	95% CI (p50)	σ	p95	p99
Scan	1018.1	807.8	[726.2, 902.6]	788.0	2656.9	4459.9
Connect	3412.9	2688.5	[2654.3, 2962.4]	1218.2	5921.6	7947.6
Key exchange	437.7	399.6	[399.3, 400.3]	83.9	578.0	846.6
Command encrypt	0.091	0.069	[0.067, 0.072]	0.047	0.156	0.312
BLE write	90.3	89.0	[88.8, 89.2]	7.5	90.8	134.5
ACK wait	90.9	90.1	[90.09, 90.13]	6.3	91.2	135.1
Command→ACK	181.3	179.5	[179.2, 179.6]	9.7	181.3	224.9
Total	5050.1	4624.1	[4489.8, 4826.9]	1502.0	7774.5	10384.6

E. Steady State and Cold Start Command Latency

The latency experiments distinguish the normal steady state case from cold start operation. In steady state, the BLE connection and CipherChannel state have already been established; the client repeatedly encrypts a STAND_UP command, writes it over GATT, and waits for the ACK. In cold start operation, each trial repeats scanning, connection, key exchange, command encryption, BLE write, and ACK waiting.

In the laptop-client experiments, command latency is measured from the authenticated CipherChannel uplink command to the gateway timing ACK used to mark completion of the trial. This ACK is part of the measurement harness rather than the exoskeleton runtime interface. For the physical cane endpoint, the ESP32 hardware-in-the-loop run uses an encrypted ACK over the cane CipherChannel context, allowing the endpoint to measure command-to-ACK latency on its own clock without subtracting unsynchronized ESP32 and gateway timestamps.

Table II reports the steady state result, measured with the laptop client and Raspberry Pi gateway placed approximately 1 m apart in direct line of sight. Across 1000 trials, all commands succeeded. The median client-observed command-to-ACK latency was 169.6 ms, with p95 of 170.7 ms and p99 of 214.7 ms. Client side AES-GCM encryption had 0.215 ms median latency and 0.408 ms p99 latency, whereas BLE write and ACK waiting dominated the observed command path.

Table III reports the cold start result. Across 500 trials, all commands succeeded. The total cold start path had 4.62s median latency and 10.38s p99 latency. The command-to-ACK part of the same cold start trials had 179.5 ms median and 224.9 ms p99 latency. The difference shows that scan, connection establishment, and key exchange dominate cold start operation, while the encrypted command-to-ACK exchange remains in the same order of magnitude as the steady state case.

TABLE IV
DTLS 1.2 PSK COLD-START SECURE CHANNEL BASELINE OVER UDP/IP WI-FI (MS, $N = 500$, 498 SUCCEEDED)

Phase	p50	95% CI (p50)	Mean	p90	p95	p99	Max
Socket setup	0.03	[0.029, 0.030]	0.04	0.04	0.20	0.25	0.29
DTLS handshake (PSK)	22.52	[19.86, 26.51]	118.52	235.25	502.75	1181.85	4722.85
Command→ACK	3.69	[3.39, 3.94]	30.81	59.88	114.69	478.14	2722.71
Total	**27.62**	**[24.59, 33.08]**	**149.39**	**329.73**	**740.15**	**1424.14**	**5686.97**

F. DTLS Baseline Comparison

To contextualize the CipherChannel cold-start latency, we measured the cold-start round-trip time of a standard DTLS 1.2 PSK secure channel between the same laptop and Raspberry Pi hardware over UDP/IP (Wi-Fi). Each trial was a fully independent DTLS session: new UDP socket, full PSK handshake, one command datagram, one ACK datagram, and session close. No session resumption was used. The ciphersuite was TLS_PSK_WITH_AES_128_GCM_SHA256 (mbedTLS 3.6 on the laptop, mbedTLS 2.28 on the RPi). Table IV reports results for 500 trials (498 succeeded; 2 failed due to Wi-Fi packet loss triggering DTLS retransmission timeouts).

The DTLS baseline shows that a standard PSK secure channel handshake can be fast at the median on a Wi-Fi LAN: the total cold-start median was 27.62 ms and the median handshake cost was 22.52 ms . However, the distribution was strongly skewed. The mean total latency was 149.39 ms , more than five times the median, and the maximum observed total latency reached 5686.97 ms . The handshake phase was the dominant contributor to this tail, with a 1181.85 ms p99 and a 4722.85 ms maximum. The two failed trials were recorded as retransmission timeouts, producing a 99.6% success rate. This baseline is not intended to replace a BLE/GATT-specific secure-transport comparison. Rather, it provides a reference point for the cold-start cost of a standard PSK-based secure channel on the same host and gateway hardware. A DTLS-over-GATT design would require a transport adaptation layer to map DTLS records onto ATT/GATT writes and notifications, including fragmentation, reassembly, retransmission policy, and timeout handling. Such a design point is orthogonal to CipherChannel's command-admission mechanism and is left to future work.

G. Gateway Internal Processing Cost

Table V reports gateway-internal processing for 1000 laptop-originated steady-state commands. The measured interval begins inside the GATT characteristic-write callback and ends when the ACK characteristic value has been set. AES-256-GCM authentication and decryption had 1.70 ms median and 1.98 ms p99 latency. Freshness and direction-parity validation were below 0.005 ms at p99. The durable receive-counter write had 14.21 ms median and 101.87 ms p99 latency, making crash-safe persistence the dominant local gateway cost. This shows that CipherChannel cryptographic processing is not the limiting factor in the evaluated gateway path; instead, the main implementation trade-off is the cost of synchronous durable replay-state updates.

TABLE V
GATEWAY-SIDE RECEIVE-TO-ACK PROCESSING BREAKDOWN FOR LAPTOP-ORIGINATED COMMANDS (MS, $N = 1000$)

Phase	μ	p50	95% CI (p50)	σ	p95	p99
Packet parse	0.0295	0.0289	[0.0288, 0.0290]	0.0105	0.0314	0.0420
Freshness + direction validation	0.0032	0.0032	[0.0032, 0.0032]	0.0031	0.0034	0.0045
AES-256-GCM auth + decryption	1.7082	1.6989	[1.6982, 1.7003]	0.2089	1.8719	1.9765
Receive counter fsync	20.0119	14.2125	[14.1737, 14.2594]	16.5830	30.5376	101.8685
JSON parse + action identification	0.2955	0.2956	[0.2953, 0.2959]	0.0248	0.3360	0.3474
ACK characteristic update	0.4918	0.4910	[0.4909, 0.4913]	0.0489	0.5629	0.5907
Gateway total	**22.5402**	**16.8113**	**[16.7657, 16.8567]**	**16.5719**	**33.2315**	**104.4857**

To confirm that the same gateway path is usable by the physical cane endpoint, we also ran an ESP32 hardware-in-the-loop command experiment. The ESP32 connected as a BLE central, obtained its endpoint-specific cane operational key, and issued sequential encrypted `STOP` commands over the cane characteristic, starting each trial only after receiving and decrypting the previous encrypted ACK. In the current run, all 502 aligned ESP32 trials completed successfully. The ESP32-measured command-to-ACK round-trip latency was 145.03 ms median, 195.98 ms p95, and 295.15 ms p99; the ESP32 local CipherChannel send and BLE write-call medians were 8.65 ms and 1.62 ms, respectively. The corresponding gateway receive-to-ACK-set interval for the cane path had 36.64 ms median and 213.75 ms p99 latency. This hardware-in-the-loop result validates the physical ESP32 cane as an interoperable CipherChannel endpoint, while the laptop-originated measurements remain the controlled baseline used for detailed gateway-phase attribution.

The same ESP32 run also measured setup costs on the physical cane endpoint. Connection setup took 3.205,s, consisting of 1.332s BLE connection time, 1.578s service discovery, 4.632 ms characteristic discovery, and 289.899 ms notification registration. Cane key exchange took 350.386,ms, dominated by a 340.908 ms wait for the encrypted key response. These setup measurements are reported descriptively because they come from a single instrumented run rather than a distribution of cold-start trials.

H. Key Exchange Latency

Table VI reports 500 key exchange trials. Each trial includes scan, connection, `REQUEST_KEY` write, server processing, client decryption of the operational key, and client channel-state initialization. All 500 trials succeeded. The key exchange portion from request write to decrypted key had 443.5 ms median latency and 937.9 ms p99 latency. This figure characterizes the same request-to-key span reported as the "Key exchange" phase in the cold-start experiment (Table III, 399.6 ms median); the two were collected in separate runs, and the difference reflects run-to-run variation in BLE request/response scheduling rather than a change in protocol behavior. Client side decryption and state initialization were sub-millisecond at p99; the measured key exchange latency is therefore dominated by BLE request/response timing and server-side publication of the encrypted key material. Physical button latency is excluded because the benchmark provisioning gate was used.

TABLE VI
KEY EXCHANGE LATENCY (MS, $N = 500$)

Phase	μ	p50	95% CI (p50)	σ	p95	p99
Scan	991.7	741.7	[653.9, 833.4]	832.4	2581.5	3930.5
Connect	3674.0	3323.7	[2769.4, 3355.6]	1414.3	6639.7	8033.0
`REQUEST_KEY` write	363.3	352.6	[352.2, 352.9]	92.6	488.3	847.4
Server process	92.2	90.9	[90.7, 91.0]	8.7	92.4	136.3
Client decrypt key	0.303	0.238	[0.220, 0.263]	0.639	0.336	0.513
State initialization	0.028	0.019	[0.018, 0.021]	0.021	0.047	0.129
KEX total	455.8	443.5	[443.1, 444.0]	93.1	578.7	937.9

TABLE VII
DISTANCE SENSITIVITY WITH FULL RECONNECT PER TRIAL (MS, $N = 200$ PER CONDITION)

Condition	μ	p50	95% CI (p50)	σ	p95	p99	Max	Successes
0.5 m LOS	4481.9	4303.7	[3989.2, 4574.1]	1334.8	7051.8	8217.7	8444.4	200/200
1 m LOS	4468.4	4258.3	[4011.2, 4483.4]	1316.0	6823.8	7999.6	8804.1	200/200
2 m LOS	4329.0	4124.2	[3988.6, 4281.8]	1302.1	6646.8	8713.2	9164.2	200/200
4 m LOS	5002.7	4731.5	[4551.7, 4979.1]	1557.6	7560.2	9666.9	12988.4	200/200
7 m wall	5561.2	4798.5	[4529.7, 5204.6]	2359.1	10624.3	12686.4	16138.9	200/200

TABLE VIII
DISTANCE SENSITIVITY IN STEADY STATE BLE OPERATION (MS, $N = 500$ PER CONDITION)

Condition	μ	p50	95% CI (p50)	σ	p95	p99	Max	Succ.
1 m LOS	170.8	169.3	[169.30, 169.34]	8.4	170.2	214.2	258.1	500/500
2 m LOS	170.3	169.3	[169.31, 169.35]	7.2	170.2	214.3	259.6	500/500
4 m LOS	171.8	169.3	[169.29, 169.36]	10.5	211.4	214.4	258.5	500/500
7 m wall	182.0	169.4	[169.34, 169.42]	25.9	215.1	259.6	350.5	500/500

I. Distance and Obstruction Sensitivity

Two distance experiments were conducted. The first repeated full reconnect operation across five conditions: 0.5 m line of sight, 1 m line of sight, 2 m line of sight, 4 m line of sight, and 7 m with a wall. Each condition used 200 trials, for 1000 total reconnect trials. All 1000 trials succeeded, with no failures at scan, connection, key exchange, write, or ACK stages.

The second distance experiment kept an established connection open and measured sustained write/ACK latency across 1 m, 2 m, 4 m, and 7 m-wall conditions, with 500 trials per condition. All 2000 trials succeeded. The median command-to-ACK latency remained approximately 169.3–169.4 ms across all conditions, while the 7 m-wall condition increased the mean and tail latency, reaching 259.6 ms p99 and 350.5 ms maximum latency. These medians are consistent with the 169.6 ms steady-state median in Table II, which was collected in a separate run; the sub-millisecond differences reflect run-to-run variation rather than a distance effect.

A final exploratory run was then performed to observe behavior near the edge of the tested BLE range. In this run, the client started at approximately 7 m and progressively moved farther from the gateway until failure, reaching approximately 10–11 m through two walls near the first connection-loss event. The first 43 commands were acknowledged successfully. Trial 44 completed the GATT write but failed while reading the ACK, and trial 45 reported connection loss. Subsequent logged attempts occurred after the connection had already been lost and are therefore not counted as independent command

TABLE IX
ESP32 CANE-CONTROLLER FIRMWARE RESOURCE FOOTPRINT

Metric	Observed value
Total flash / ROM usage with CipherChannel	1,160,949 B / 1,310,720 B (88.5%)
Total flash / ROM usage without CipherChannel	1,147,437 B / 1,310,720 B (87.5%)
Estimated CipherChannel flash increment	11,416 B, i.e., approx. 11.16 KiB
Build-reported dynamic RAM with CipherChannel	38,748 B / 327,680 B (11.8%)
Minimum free heap during operation	118,864 B
Minimum free stack during operation	5,448 B

Note: Flash and global dynamic-RAM values are taken from Arduino build output. Heap and stack values were measured at runtime during BLE and CipherChannel operation using ESP32 heap instrumentation and the FreeRTOS stack high-water mark. The CipherChannel flash increment was estimated by compiling the same firmware with CipherChannel enabled and disabled.

delivery trials.

These results refine the interpretation of the distance experiments. CipherChannel maintained reliable command delivery in the fixed 1–7 m conditions and remained operational during the progressive two wall walk-back until the edge of the tested BLE range. Near that edge, latency variance increased substantially and the first observed failure occurred as an ACK read error, followed by connection loss at approximately 10 m through two walls. This indicates that the limiting factor in the tested configuration was BLE link availability rather than CipherChannel authentication or replay validation.

J. ESP32 Resource Footprint

Because the auxiliary cane controller is implemented on an ESP32-class microcontroller, we report the embedded resource footprint of the Arduino/mbedTLS implementation in Table IX. Prior work on low-end IoT cryptographic hardware shows that cryptographic cost is strongly platform- and acceleration-dependent, so implementation-level resource measurements are necessary rather than inferable from the protocol design alone [34]. Flash and dynamic RAM usage were obtained from the Arduino build output. Runtime stack availability was estimated using the FreeRTOS stack high-water mark reported during the BLE/CipherChannel execution path. Power consumption was not measured in this work. Espressif reports a typical ESP32 BT/BLE transmit current of 130 mA at 0 dBm output power, with BT/BLE receive current in the 95–100 mA range under the datasheet measurement conditions [52].

VII. DISCUSSION

A. Interpretation of the Main Results

The evaluation supports the central claim of the paper: BLE connection establishment should not be treated as command authorization for a safety relevant exoskeleton command channel. Under the tested conditions, CipherChannel rejected all replay, tampering, stale counter, wrong key, reflection, and cross endpoint injection attempts before runtime forwarding. The results therefore support the protocol's admission-control argument: an attacker who can observe, replay, reorder, or modify BLE traffic still lacks a sufficient condition for command acceptance unless the packet also authenticates under the correct operational key and carries a fresh counter nonce with the expected direction parity.

The latency results also support the practical feasibility claim, but with a narrower interpretation than a general real-time-control claim. CipherChannel is appropriate for sparse supervisory commands such as `STAND_UP`, `START_WALKING`, `TAKE_STEP`, and `STOP_ACTION`. It is not evaluated as a high-rate joint-control or servo-loop security mechanism. In steady state BLE operation, the median command-to-ACK latency was 169.6 ms and the p99 latency was 214.7 ms . These values are at or below the mean simple reaction time reported for visual stimuli under calibrated conditions [51], indicating that the cryptographic round-trip is small relative to the human perceptual-motor loop for a clinician- or user-initiated command; this comparison should nonetheless be interpreted cautiously. It supports plausibility for clinician- or user-initiated supervisory control; it does not establish suitability for closed-loop motion control.

B. Latency Trade-offs

The main latency trade-off is not AES-GCM itself. In the steady-state experiment, client-side encryption had 0.215 ms median latency, while gateway-side AES-256-GCM authentication and decryption had 1.70 ms median latency. The command-to-ACK path was instead dominated by BLE write/ACK timing and by crash-safe receive-counter persistence. The fsync-based receive-counter update had 14.21 ms median and 101.87 ms p99 latency, making synchronous durable replay-state storage the main local gateway cost.

This cost is acceptable for sparse supervisory commands, but production implementations should choose persistent storage deliberately. Possible approaches include hardware-backed monotonic counters, journaled key-value storage, wear-aware non-volatile memory, or an audited batching policy, provided the core invariant is preserved: an accepted receive counter must be made durable before the plaintext command is released to the runtime.

To isolate the persistence mechanism, we also measured the unmodified `_write_state()` routine on the laptop host over 1000 calls using two state-directory backends. An ext4/NVMe-backed directory produced 2.21 ms median and 3.97 ms p99 latency, whereas a tmpfs directory under `/dev/shm` produced 0.0075 ms median and 0.013 ms p99 latency, an approximately 295× median reduction. This host-side measurement is not directly comparable to the Raspberry Pi gateway, but it confirms that the durability barrier, rather than the cryptographic primitive, dominates the persistence cost. Plain tmpfs is not suitable for CipherChannel state because it does not survive power loss; achieving similar latency while preserving crash safety would require a durable low-latency storage mechanism such as NVRAM, battery-backed RAM, or an equivalent platform-specific facility.

C. Physical Presence Requirement as a Safety Control

The provisioning gate is intended as an operational safety control in addition to BLE pairing. Before an endpoint can obtain operational key material, an authorized operator must explicitly open a short provisioning window, for example through a physical button press on the middleware device. This limits unattended or remote enrolment and provides an out-of-band authorization step suited to a clinical or supervised exoskeleton setting. The implemented tests validate the software state machine associated with this gate, including one-shot provisioning windows, timeout handling, endpoint separation, and rejection when the gate is closed. The physical button interaction itself is a simple hardware gating condition; full hardware-in-the-loop validation of the button and GPIO path remains part of system testing.

D. Distance, Obstruction, and Deployment Constraints

The distance results suggest that CipherChannel's cryptographic framing was not the limiting factor in the tested deployment envelope. Across the fixed-distance line-of-sight and wall obstructed conditions, command admission remained reliable and median command-to-ACK latency stayed in the same operating range. Obstruction mainly affected variance and tail latency rather than the median path. The progressive two wall walk-back run further indicates that the practical limit was BLE link quality: the system continued to complete authenticated command-to-ACK exchanges until the connection eventually degraded and failed near the edge of coverage. Therefore, deployment constraints for this implementation are dominated by BLE radio conditions, wall attenuation, and connection stability, rather than by AES-GCM processing or protocol framing overhead.

E. Limitations

The evaluated prototype has several implementation and deployment limitations. CipherChannel protects command confidentiality, integrity, authentication, replay resistance, and crash-safe counter persistence, but it does not by itself solve endpoint compromise, radio-level denial of service, or broader system safety policy. The current implementation also uses provisioning assumptions and key-management choices that are appropriate for a controlled prototype but should be strengthened in production through hardware-backed key storage, revocation, audited reprovisioning, and authenticated ephemeral key establishment. Finally, the measured latency results characterize the tested Raspberry Pi, BlueZ/Bleak, and ESP32 stacks; deployments on other BLE controllers, mobile operating systems, connection parameters, or persistent-storage media should remeasure the command path. These limitations do not affect the central result: under the tested conditions, CipherChannel provides a durable above-GATT command-admission boundary and rejects unauthenticated, stale, replayed, or cross-endpoint commands before runtime forwarding.

F. Regulatory Alignment

CipherChannel is not a complete regulatory compliance framework, but it addresses protocol level concerns that are relevant to current medical device cybersecurity expectations. FDA's 2023 guidance identifies secure development, threat modeling, SBOM documentation, vulnerability management, and cryptographic protection among the cybersecurity considerations for medical device submissions [27]. IEC 81001-5-1:2021 and IEC TR 60601-4-5:2021 provide lifecycle and technical-security guidance for health software and medical electrical equipment [28]–[30]. Within this narrower scope, CipherChannel contributes an application-layer communication control based on AES-256-GCM, persistent replay protection, endpoint isolation, and physically gated key provisioning before commands reach the exoskeleton runtime.

G. Future Work

Future work will extend CipherChannel in four directions. First, the provisioning design should be strengthened by replacing the static bootstrap key with authenticated ephemeral key establishment, endpoint-specific operational keys, and hardware-backed key storage on the gateway and controller devices. Second, the BLE transport layer should be made more portable by adding explicit application level fragmentation and reassembly above GATT, so that CipherChannel frames are not dependent on stack-specific long write behavior in BlueZ, mobile operating systems, or microcontroller BLE implementations. Third, availability and resilience should be evaluated under adversarial radio and protocol conditions, including GATT flooding, packet suppression, repeated reconnect attempts, and multi-endpoint contention, with explicit timeout, retry, and fail safe policies. Fourth, the protocol should be validated in longer clinical style workflows with multiple users, repeated provisioning events, endpoint replacement, and realistic supervisory command sequences, so that the cryptographic boundary can be evaluated together with usability, operational safety, and medical device deployment constraints.

VIII. Conclusion

This paper presented CipherChannel, an application-layer authenticated encryption protocol for securing BLE based command and control of a medical lower-limb exoskeleton. By combining AES-256-GCM, persistent counter nonces, replay protection, endpoint isolation, and physically gated key provisioning, CipherChannel separates BLE connectivity from actual command authorization. The evaluation showed that the protocol correctly rejected replay, tampering, stale counter, wrong key, reflection, reordering, and cross endpoint injection attempts, while maintaining practical command-to-ACK latency for encrypted supervisory commands and for a physical ESP32 cane endpoint. These results suggest that CipherChannel can provide a replay resistant security boundary for sparse exoskeleton commands without modifying the proprietary runtime, provided that deployment-specific BLE conditions, persistent-storage behavior, endpoint key management, and fail-safe availability controls are engineered and validated for the target clinical environment.

REFERENCES

[1] D. Antonioli, N. O. Tippenhauer, and K. Rasmussen, "Key Negotiation Downgrade Attacks on Bluetooth and Bluetooth Low Energy," *ACM Transactions on Privacy and Security*, vol. 23, no. 3, art. 14, pp. 1–28, 2020, doi: https://doi.org/10.1145/3394497.

[2] D. Antonioli, N. O. Tippenhauer, and K. Rasmussen, "BIAS: Bluetooth Impersonation AttackS," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 549–562, 2020, doi: https://doi.org/10.1109/SP40000.2020.00093.

[3] J. Wu, Y. Nan, V. Kumar, D. J. Tian, A. Bianchi, M. Payer, and D. Xu, "BLESA: Spoofing Attacks against Reconnections in Bluetooth Low Energy," in *Proc. 14th USENIX Workshop on Offensive Technologies (WOOT)*, 2020. [Online]. Available: https://www.usenix.org/system/files/woot20-paper-wu-updated.pdf

[4] M. E. Garbelini, C. Wang, S. Chattopadhyay, S. Sun, and E. Kurniawan, "SweynTooth: Unleashing Mayhem over Bluetooth Low Energy," in *Proc. USENIX Annual Technical Conference (ATC '20)*, pp. 911–925, 2020. [Online]. Available: https://www.usenix.org/conference/atc20/presentation/garbelini

[5] R. Cayre, F. Galtier, G. Auriol, V. Nicomette, M. Kaaniche, and G. Marconato, "InjectaBLE: Injecting Malicious Traffic into Established Bluetooth Low Energy Connections," in *Proc. IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 388–399, 2021, doi: https://doi.org/10.1109/DSN48987.2021.00050.

[6] D. Antonioli, "BLUFFS: Bluetooth Forward and Future Secrecy Attacks and Defenses," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 636–650, 2023, doi: https://doi.org/10.1145/3576915.3623066.

[7] Y. Zhang, J. Weng, R. Dey, Y. Jin, Z. Lin, and X. Fu, "Breaking Secure Pairing of Bluetooth Low Energy Using Downgrade Attacks," in *Proc. 29th USENIX Security Symposium*, pp. 37–54, 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/zhang-yue

[8] M. von Tschirschnitz, L. Peuckert, F. Franzen, and J. Grossklags, "Method Confusion Attack on Bluetooth Pairing," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 1332–1347, 2021, doi: https://doi.org/10.1109/SP40001.2021.00013.

[9] A. Barua, M. A. Al Alamin, M. S. Hossain, and E. Hossain, "Security and Privacy Threats for Bluetooth Low Energy in IoT and Wearable Devices: A Comprehensive Survey," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 251–281, 2022, doi: https://doi.org/10.1109/OJCOMS.2022.3149732.

[10] Y. K. Peker, G. Bello, and A. J. Perez, "On the Security of Bluetooth Low Energy in Two Consumer Wearable Heart Rate Monitors/Sensing Devices," *Sensors*, vol. 22, no. 3, art. 988, 2022, doi: https://doi.org/10.3390/s22030988.

[11] S. Sevier and A. Tekeoglu, "Analyzing the Security of Bluetooth Low Energy," in *Proc. International Conference on Electronics, Information, and Communication (ICEIC)*, pp. 1–5, 2019, doi: https://doi.org/10.23919/ELINFOCOM.2019.8706457.

[12] G. Amirkhanova, S. Ismailov, A. Amirkhanov, S. Adilzhanova, M. Zhasuzakova, and S. Chen, "A Lightweight, End-to-End Encrypted Data Pipeline for IIoT: An AES-GCM Implementation for ESP32, MQTT, and Raspberry Pi," *Information*, vol. 17, no. 1, art. 33, 2026, doi: https://doi.org/10.3390/info17010033.

[13] H. Rakshit, R. Bhandari, and S. Banerjee, "Lightweight Session-Key Rekeying Framework for Secure IoT-Edge Communication," *arXiv preprint arXiv:2511.02924*, 2025. [Online]. Available: https://arxiv.org/abs/2511.02924

[14] R. Baud, A. R. Manzoori, A. Ijspeert, and M. Bouri, "Review of Control Strategies for lower limb Exoskeletons to Assist Gait," *Journal of NeuroEngineering and Rehabilitation*, vol. 18, art. 119, 2021, doi: https://doi.org/10.1186/s12984-021-00906-3.

[15] M. R. Tucker, J. Olivier, A. Pagel, H. Bleuler, M. Bouri, O. Lambercy, J. d. R. Millán, R. Riener, H. Vallery, and R. Gassert, "Control Strategies for Active Lower Extremity Prosthetics and Orthotics: A Review," *Journal of NeuroEngineering and Rehabilitation*, vol. 12, art. 1, 2015, doi: https://doi.org/10.1186/1743-0003-12-1.

[16] D. Pinto-Fernandez, D. Torricelli, M. d. C. Sanchez-Villamanan, F. Aller, K. Mombaur, R. Conti, N. Vitiello, J. C. Moreno, and J. L. Pons, "Performance Evaluation of Lower Limb Exoskeletons: A Systematic Review," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 28, no. 7, pp. 1573–1583, 2020, doi: https://doi.org/10.1109/TNSRE.2020.2989481.

[17] R. B. van Dijsseldonk, H. Rijken, I. J. W. van Nes, H. van de Meent, and N. L. W. Keijsers, "Predictors of exoskeleton motor learning in spinal cord injured patients," *Disability and Rehabilitation*, vol. 43, no. 12, pp. 1742–1748, 2021, doi: https://doi.org/10.1080/09638288.2019.1689578.

[18] H. Y. Lee, J. H. Park, and T.-W. Kim, "Comparisons between Locomat and Walkbot robotic gait training regarding balance and lower extremity function among non-ambulatory chronic acquired brain injury survivors," *Medicine*, vol. 100, no. 18, art. e25125, 2021, doi: https://doi.org/10.1097/MD.0000000000025125.

[19] C. Cumplido-Trasmonte, J. Ramos-Rojas, E. Delgado-Castillejo, E. Garcés-Castellote, G. Puyuelo-Quintana, M. A. Destarac-Eguizabal, E. Barquín-Santos, A. Plaza-Flores, M. Hernández-Melero, A. Gutiérrez-Ayala, and others, "Effects of ATLAS 2030 Gait Exoskeleton on Strength and Range of Motion in Children with Spinal Muscular Atrophy II: A Case Series," *Journal of NeuroEngineering and Rehabilitation*, vol. 19, art. 75, 2022, doi: https://doi.org/10.1186/s12984-022-01055-x.

[20] R. Andersson, M. Cronhjort, and J. Chilo, "Wireless PID-Based Control for a Single-Legged Rehabilitation Exoskeleton," *Machines*, vol. 12, no. 11, art. 745, 2024, doi: https://doi.org/10.3390/machines12110745.

[21] A. Ozhiken, G. Sergazin, K. Ozhikenov, H. Wang, N. Zhetenbayev, G. Tursunbayeva, A. Nurmangaliyev, and A. Uzbekbayev, "Research and Experimental Testing of a Remotely Controlled Ankle Rehabilitation Exoskeleton Prototype," *Sensors*, vol. 25, no. 21, art. 6784, 2025, doi: https://doi.org/10.3390/s25216784.

[22] N. J. Wilhelm, S. Haddadin, J. J. Lang, C. Micheler, F. Hinterwimmer, A. Reiners, R. Burgkart, and C. Glowalla, "Development of an Exoskeleton Platform of the Finger for Objective Patient Monitoring in Rehabilitation," *Sensors*, vol. 22, no. 13, art. 4804, 2022, doi: https://doi.org/10.3390/s22134804.

[23] A. Wang, J. Dong, R. Teng, P. Liu, X. Yue, and X. Zhang, "A Hierarchical Distributed Control System Design for Lower Limb Rehabilitation Robot," *Technologies*, vol. 13, no. 10, art. 462, 2025, doi: https://doi.org/10.3390/technologies13100462.

[24] H. Chi, L. Wu, X. Du, Q. Zeng, and P. Ratazzi, "e-SAFE: Secure, Efficient and Forensics-Enabled Access to Implantable Medical Devices," in *Proc. IEEE Conference on Communications and Network Security (CNS)*, pp. 1–9, 2018, doi: https://doi.org/10.1109/CNS.2018.8433213.

[25] E. Marin, D. Singelée, B. Yang, V. Volskiy, G. A. E. Vandenbosch, B. Nuttin, and B. Preneel, "Securing Wireless Neurostimulators," in *Proc. ACM Conference on Data and Application Security and Privacy (CODASPY)*, pp. 287–298, 2018, doi: https://doi.org/10.1145/3176258.3176310.

[26] G. Zheng, G. Zhang, W. Yang, C. Valli, R. Shankaran, and M. A. Orgun, "From WannaCry to WannaDie: Security Trade-offs and Design for Implantable Medical Devices," in *Proc. 17th International Symposium on Communications and Information Technologies (ISCIT)*, pp. 1–5, 2017, doi: https://doi.org/10.1109/ISCIT.2017.8261228.

[27] U.S. Food and Drug Administration, "Cybersecurity in Medical Devices: Quality System Considerations and Content of Premarket Submissions," final guidance, Sep. 2023. [Online]. Available: https://www.fda.gov/medical-devices/digital-health-center-excellence/cybersecurity

[28] International Electrotechnical Commission, *IEC 80001-1:2021, Application of Risk Management for IT-Networks Incorporating Medical Devices—Part 1: Safety, Effectiveness and Security in the Implementation and Use of Connected Medical Devices or Connected Health Software*, IEC, 2021.

[29] International Electrotechnical Commission, *IEC TR 60601-4-5:2021, Medical Electrical Equipment—Part 4-5: Guidance and Interpretation—Safety-Related Technical Security Specifications*, IEC, 2021.

[30] International Electrotechnical Commission, *IEC 81001-5-1:2021, Health Software and Health IT Systems Safety, Effectiveness and Security—Part 5-1: Security—Activities in the Product Life Cycle*, IEC, 2021.

[31] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, National Institute of Standards and Technology, Nov. 2007, doi: https://doi.org/10.6028/NIST.SP.800-38D.

[32] B. Blanchet, "Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif," *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, Oct. 2016, doi: https://doi.org/10.1561/3300000004.

[33] W. Riddle, G. Quan, S. Salerno, E. Mendez, V. Formicola, N. El-Naga, and M. El-Hadedy, "AESPIE: Raspberry Pi AES Performance Evaluation Using Image Encryption in C and Python," in *Applications of Remote Sensing and GIS Based on an Innovative Vision*, A. A. Gad, D. Elfiky, A. Negm, and S. Elbeih, Eds. Cham, Switzerland: Springer, pp. 257–265, 2023, doi: https://doi.org/10.1007/978-3-031-40447-4_30.

[34] P. Kietzmann, L. Boeckmann, L. Lanzieri, T. C. Schmidt, and M. Wählisch, "A Performance Study of Crypto-Hardware in the Low-End IoT," in *Proc. International Conference on Embedded Wireless Systems and Networks (EWSN)*, pp. 79–90, 2021. [Online]. Available: https://dl.acm.org/doi/10.5555/3451271.3451279

[35] S. S. Hassan, S. D. Bibon, M. S. Hossain, and M. Atiquzzaman, "Security Threats in Bluetooth Technology," *Computers & Security*, vol. 74, pp. 308–322, 2018, doi: https://doi.org/10.1016/j.cose.2017.03.008.

[36] M. Cäsar, T. Pawelke, J. Steffan, and G. Terhorst, "A Survey on Bluetooth Low Energy Security and Privacy," *Computer Networks*, vol. 205, art. 108712, 2022, doi: https://doi.org/10.1016/j.comnet.2021.108712.

[37] Q. Zhang, Z. Liang, and Z. Cai, "Developing a New Security Framework for Bluetooth Low Energy Devices," *Computers, Materials & Continua*, vol. 59, no. 2, pp. 457–471, 2019. doi: https://doi.org/10.32604/cmc.2019.03758.

[38] S.-C. Cha, K.-H. Yeh, and J.-F. Chen, "Toward a Robust Security Paradigm for Bluetooth Low Energy-Based Smart Objects in the Internet-of-Things," *Sensors*, vol. 17, no. 10, p. 2348, 2017. doi: https://doi.org/10.3390/s17102348.

[39] M. R. Ghori, T.-C. Wan, and G. C. Sodhy, "Bluetooth Low Energy Mesh Networks: Survey of Communication and Security Protocols," *Sensors*, vol. 20, no. 12, p. 3590, 2020. doi: https://doi.org/10.3390/s20123590.

[40] H. Greß, B. Krüger, and E. Tischhauser, "The Newer, the More Secure? Standards-Compliant Bluetooth Low Energy Man-in-the-Middle Attacks on Fitness Trackers," *Sensors*, vol. 25, no. 6, p. 1815, 2025. doi: https://doi.org/10.3390/s25061815.

[41] H. Mrabet, S. Belguith, A. Alhomoud, and A. Jemai, "A Survey of IoT Security Based on a Layered Architecture of Sensing and Data Analysis," *Sensors*, vol. 20, no. 13, p. 3625, 2020. doi: https://doi.org/10.3390/s20133625.

[42] P. G. Vinoj, S. Jacob, V. G. Menon, S. Rajesh, and M. R. Khosravi, "Brain-Controlled Adaptive Lower Limb Exoskeleton for Rehabilitation of Post-Stroke Paralyzed," *IEEE Access*, vol. 7, pp. 132628–132648, 2019. doi: https://doi.org/10.1109/ACCESS.2019.2921375.

[43] M. Adil, M. K. Khan, N. Kumar, M. Attique, A. Farouk, M. Guizani, and Z. Jin, "Healthcare Internet of Things: Security Threats, Challenges, and Future Research Directions," *IEEE Internet of Things Journal*, vol. 11, no. 11, pp. 19046–19069, Jun. 2024. doi: https://doi.org/10.1109/JIOT.2024.3360289.

[44] A. Irshad, A. Aljaedi, Z. Bassfar, S. S. Jamal, M. Usman, and S. A. Chaudhry, "FA-SMW: Fog-Driven Anonymous Lightweight Access Control for Smart Medical Wearables," *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 4275–4285, 2025. doi: https://doi.org/10.1109/JIOT.2024.3484945.

[45] J. Chen, Y. Shi, C. Yi, H. Du, J. Kang, and D. Niyato, "Generative AI-Driven Human Digital Twin in IoT Healthcare: A Comprehensive Survey," *IEEE Internet of Things Journal*, vol. 11, no. 22, pp. 34749–34773, 2024. doi: https://doi.org/10.1109/JIOT.2024.3421918.

[46] M. A. Khatun, S. F. Memon, C. Eising, and L. L. Dhirani, "Machine Learning for Healthcare-IoT Security: A Review and Risk Mitigation," *IEEE Access*, vol. 11, pp. 145869–145896, 2023. doi: https://doi.org/10.1109/ACCESS.2023.3346320.

[47] F. J. Jaime, A. Muñoz, F. Rodríguez-Gómez, and A. Jerez-Calero, "Strengthening Privacy and Data Security in Biomedical Microelectromechanical Systems by IoT Communication Security and Protection in Smart Healthcare," *Sensors*, vol. 23, no. 21, Art. no. 8944, 2023. doi: https://doi.org/10.3390/s23218944.

[48] A. A. Devi, E. S. Babu, R. S. Rathore, R. H. Jhaveri, and F. Benedetto, "Blockchain-Based Resilient Pairing and Bonding of BLE Devices Using Deep Reinforcement Learning," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 2, pp. 4415–4429, May 2025. doi: https://doi.org/10.1109/TCE.2024.3516756.

[49] W. Hadrich and A. Sikora, "A Combination of Bluetooth Physical Layer Key Generation and Physically Unclonable Functions for Lightweight Security in IoT Edge Nodes," in *Proc. 2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8. doi: https://doi.org/10.1109/ETFA65518.2025.11205627.

[50] F. De Santis, A. Schauer, and G. Sigl, "ChaCha20-Poly1305 Authenticated Encryption for High-Speed Embedded IoT Applications," in *Proc. Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2017, pp. 692–697. doi: https://doi.org/10.23919/DATE.2017.7927078.

[51] D. L. Woods, J. M. Wyma, E. W. Yund, T. J. Herron, and B. Reed, "Factors influencing the latency of simple reaction time," *Front. Hum. Neurosci.*, vol. 9, art. no. 131, Mar. 2015, doi: https://doi.org/10.3389/fnhum.2015.00131.

[52] Espressif Systems, "ESP32 Series Datasheet: 2.4 GHz Wi-Fi + Bluetooth® + Bluetooth LE SoC," Version 5.2, 2025. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf